

Recursive Descent Parser

A **Recursive Descent Parser** is a **top-down parser** used in compiler design.

- It starts parsing from the **start symbol** of the grammar.
- Each **non-terminal** in the grammar is represented by a **function**.
- These functions may **call themselves recursively**.
- The parser reads the input from **left to right**.

Grammar Rule $\rightarrow$ Function $\rightarrow$ Match input $\rightarrow$ Call another function

1. Example Grammar

$E \rightarrow i E'$
 $E' \rightarrow + i E' \mid \epsilon$

- $E, E' \rightarrow$ Non-terminals
- $i \rightarrow$ Identifier
- $+$ $\rightarrow$ Addition operator
- $\epsilon \rightarrow$ Empty / nothing

This grammar can recognize:

i
 $i+i$
 $i+i+i$

2. Convert Grammar into Functions

Each non-terminal becomes a function.

For $E \rightarrow i E'$

```
E()  
{  
    match(i);  
    E'();  
}
```

For $E' \rightarrow + i E' \mid \epsilon$

```
E'()  
{  
    if (current input is '+')  
    {  
        match(+);  
        match(i);  
        E'();  
    }  
    else  
        return;  
}
```

So, grammar rules are converted into functions.

3. Parsing Example: $i + i$

Input: $i + i \$$
 $\$$ = End of input

Step 1: $E()$

$E \rightarrow i E'$

- Read $i \rightarrow$ **match**
- Call $E'()$

Step 2: $E'()$

$E' \rightarrow + i E'$

- Read $+$ $\rightarrow$ **match**
- Read $i \rightarrow$ **match**
- Call $E'()$ again

Step 3: $E'()$ again

Now input is $\$$.

- No $+$ is found
- Choose $E' \rightarrow \epsilon$
- ϵ means **nothing more to read $\rightarrow$ return**

Step 4: Result

All input is successfully consumed.

☞ $i + i$ is accepted by the parser ✓

4. Function Call Flow

The complete flow can be represented as:

```
E ()
|
|-- match i
|
└─ E' ()
    |
    |-- match +
    |-- match i
    |
    └─ E' ()
        |
        └─ ε → return
```

5. Why "Recursive Descent"?

Recursive means *A function can call itself.*

$E' () \rightarrow E' ()$

Descent

The parser starts from the **top/start symbol** and goes down through the grammar rules.

6. Important Points

- It is a **top-down parsing technique**.
- Each non-terminal is implemented as a **function**.
- Functions can call each other **recursively**.
- Input is processed from **left to right**.